Apache Hadoop Complete Architecture & Deployment Guide

16-Thread Fast Installation • Terminal Lifecycle CLI (ran-hadoop) • MapReduce WordCount Tutorial

Hadoop Version: 3.3.6 / 3.4.x	**Target OS:** WSL (Windows 10/11), Ubuntu, Debian, Kali, CentOS, RHEL, Arch
Runtime JDK: OpenJDK 11 / OpenJDK 8	**Accelerator:** aria2 (16 Concurrent Connections / Multi-Source)
Process Lifecycle: ran-hadoop (Ctrl+C & SIGHUP auto-stop)	**Verification Demo:** Distributed MapReduce WordCount on YARN

1. Executive Summary & Architecture Overview

Apache Hadoop is the bedrock framework for distributed storage (HDFS) and distributed parallel processing (YARN & MapReduce) across commodity server clusters. In a single-node **pseudo-distributed mode**, all Hadoop daemons run on a single machine (such as a Windows Subsystem for Linux (WSL) instance or Linux host) simulating a fully functional multi-node cluster with full API compatibility.

Component	Daemon Name	Function & Role in Cluster	Default Port / UI
HDFS Master	NameNode	Manages filesystem namespace, directory tree, block-to-DataNode mappings	9000 (RPC) / 9870 (Web)
HDFS Worker	DataNode	Stores and retrieves actual data blocks (typically 128MB chunks) on disk	9866 (Data) / 9864 (Web)
HDFS Checkpoint	SecondaryNameNode	Merges edit logs with fsimage periodically to prevent log explosion	9868 (Web)
YARN Master	ResourceManager	Arbitrates compute resources (CPU/RAM) across all submitted jobs	8032 (RPC) / 8088 (Web)
YARN Worker	NodeManager	Launches and monitors MapReduce task containers on the host node	8042 (Web)

2. Prerequisites & System Preparation (WSL / Any Linux)

This installation is engineered to run seamlessly inside **Windows Subsystem for Linux (WSL2)** or native Linux distributions (Ubuntu, Debian, Kali, CentOS, RHEL, Fedora, Rocky, Arch, openSUSE).

A. Setting up WSL on Windows 10 / 11 (if not already enabled):

Open Windows PowerShell as Administrator and run:

```
wsl --install
wsl --set-default-version 2
```

B. Java JDK Selection (Why OpenJDK 11 or 8?):

Apache Hadoop core, IPC serialization, and YARN node containers have strict compatibility with **OpenJDK 11** and **OpenJDK 8**. Bleeding-edge Java versions (such as Java 17, 21, or 25) enforce JVM module encapsulation, requiring numerous --add-opens reflective access flags. The installer automatically detects, verifies, and installs **OpenJDK 11** as the golden standard.

> **■ Important Tip for WSL Users**
> Inside WSL, services like SSH daemon (sshd) do not always start automatically on boot. The ran-hadoop CLI and install.sh automatically check and start sshd for you, ensuring zero connection errors.

3. One-Click Installation & 16-Thread Fast Download

Hadoop distribution tarballs exceed 700MB. Downloading over single-stream HTTP can be throttled or stall. Our installer deploys **aria2c** configured for **16 concurrent segment downloads** across mirror fallbacks, maximizing throughput and completing the download in seconds.

Download Accelerator Command Implemented in install.sh:

```
aria2c --split=16 --max-connection-per-server=16 --min-split-size=1M \
--max-concurrent-downloads=16 --file-allocation=none \
https://dlcdn.apache.org/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz
```

Step-by-Step Installation Procedure:

1. Clone the repository and enter the directory:

```
git clone <repo-url> hadoop-quickstart
cd hadoop-quickstart
```

2. Execute the one-click installer:

```
chmod +x install.sh ran-hadoop run-wordcount.sh
./install.sh
```

3. (Alternative for Windows): Double-click setup-wsl.bat in File Explorer to launch and install inside WSL with zero typing.

What install.sh Does Automatically:

- **Distro Detection:** Supports apt (Debian/Ubuntu/Kali), dnf/yum (RHEL/Fedora), pacman (Arch), zypper (openSUSE).
- **Dependency Setup:** Installs OpenJDK 11, aria2, openssh-server, rsync, and procps.
- **Passwordless SSH:** Generates RSA keys, configures ~/.ssh/authorized_keys, sets StrictHostKeyChecking no.
- **16-Thread Download:** Retrieves Hadoop 3.3.6 from official Apache CDN with multi-stream acceleration.
- **Directory Extraction:** Extracts to $HOME/hadoop, avoiding root permission locks in WSL.
- **Configuration Generation:** Builds core-site.xml, hdfs-site.xml, mapred-site.xml, yarn-site.xml, and hadoop-env.sh.
- **Storage Initialization:** Formats the HDFS NameNode automatically.
- **System Integration:** Installs ran-hadoop into /usr/local/bin and configures ~/.bashrc.

4. The 'ran-hadoop' Terminal Lifecycle CLI

Standard Hadoop utilities (start-dfs.sh, start-yarn.sh) spawn disconnected background background daemons. If a user closes their WSL or Linux window, JVM processes can stay orphaned, holding ports 9000 and 9870, locking NameNode editlogs, and draining RAM.

The custom **ran-hadoop** command solves this by providing a unified foreground cluster lifecycle manager with **automatic POSIX signal cleanup**.

Command Usage:

```
ran-hadoop
```

Key Lifecycle Features:

- **Single Command Startup:** Automatically verifies SSH, initializes NameNode if needed, and starts DFS and YARN.
- **Live Health Dashboard:** Displays running daemon PIDs and clickable browser links to Web UIs.
- **Traps Ctrl + C (SIGINT):** Intercepting Ctrl+C gracefully invokes stop-yarn.sh and stop-dfs.sh.
- **Traps Terminal Close (SIGHUP):** When the terminal window or WSL tab is closed, the operating system issues a SIGHUP (Hangup) signal to the process group. ran-hadoop traps this signal and shuts down all daemons cleanly before exiting.
- **Orphan Process Prevention:** Performs a secondary sweep to guarantee no rogue Java processes remain.

Under the Hood: POSIX Signal Trap Mechanism in ran-hadoop:

```
cleanup() {
trap '' SIGINT SIGTERM SIGHUP EXIT # Prevent recursion
echo "[ran-hadoop] Stopping YARN and HDFS daemons..."
"$HADOOP_HOME/sbin/stop-yarn.sh"
"$HADOOP_HOME/sbin/stop-dfs.sh"
exit 0
}
trap cleanup SIGINT SIGTERM SIGHUP EXIT
```

5. Step-by-Step WordCount MapReduce Tutorial

The **WordCount** job is the classic 'Hello World' of distributed computing. It demonstrates parallel tokenization across split blocks (Map phase) and aggregation of token frequencies across partitions (Reduce phase).

Phase	Input	Operation	Output Key-Value Pair
1. Input Split	Text file in HDFS	HDFS breaks file into 128MB block splits	<byte_offset, line_text>
2. Map	Line of text	Mapper tokenizes words and emits count '1'	<word, 1>
3. Shuffle / Sort	Mapped pairs	Partitions pairs by key hash and groups identical keys	<word, [1, 1, 1, ...]>
4. Reduce	Grouped values	Sums total count list for each distinct word	<word, total_count>

Executing WordCount Manually (Step-by-Step):

Step 1: While ran-hadoop is running in Terminal 1, open **Terminal 2**.

Step 2: Create an input directory inside HDFS:

```
hdfs dfs -mkdir -p /wordcount_demo/input
```

Step 3: Upload the sample text file (or any document) into HDFS:

```
hdfs dfs -put -f sample_input.txt /wordcount_demo/input/
```

Step 4: Execute the bundled MapReduce examples JAR:

```
hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-*.jar \
wordcount /wordcount_demo/input /wordcount_demo/output
```

Step 5: Inspect the word count output from HDFS:

```
hdfs dfs -cat /wordcount_demo/output/part-r-00000 | sort -k2 -nr | head -n 25
```

One-Click Execution with run-wordcount.sh:

You can also execute all the above steps automatically with a single script:

```
./run-wordcount.sh [optional_custom_textfile.txt]
```

6. Web User Interfaces & Cluster Verification

Hadoop provides rich browser-based administrative portals for monitoring storage and distributed compute jobs.

Web Portal	Browser URL	Key Metrics & Monitoring Features
HDFS NameNode UI	http://localhost:9870	DFS Used/Remaining capacity, Live DataNodes status, Block pool breakdown, Utilities > Browse the File System.
YARN ResourceManager UI	http://localhost:8088	Active/Completed MapReduce applications, NodeManager health, Memory (MB) and VCores allocated, container execution logs.
HDFS DataNode UI	http://localhost:9864	Individual node block statistics, physical disk mount utilization, and block storage reports.

Terminal Diagnostic Verification (jps):

Run jps in any terminal. A healthy cluster will show exactly these 5 daemons:

```
12345 NameNode
12456 DataNode
12567 SecondaryNameNode
12678 ResourceManager
12789 NodeManager
12890 Jps
```

7. Troubleshooting & Common Pitfalls

Symptom / Error	Root Cause	Proven Solution
Name node is in safe mode	HDFS requires a brief warmup period or block threshold check on startup.	Force leave safe mode by executing: hdfs dfsadmin -safemode leave
ssh: connect to host localhost port 22: Connection refused	OpenSSH server daemon is inactive in WSL.	Start SSH service: sudo service ssh start (install.sh and ran-hadoop do this automatically)
Container [pid=...] is running beyond virtual memory limits	WSL kernel reports different virtual memory commit ratios to YARN NodeManager.	Disabled by default in our yarn-site.xml: yarn.nodemanager.vmem-check-enabled=false
Cannot access localhost:9870 in Windows Chrome/Edge	WSL2 localhost forwarding or Windows firewall blocking bridge.	Access via WSL IP: run wsl hostname -I, then visit http://<wsl-ip>:9870 in Windows browser.
Orphan processes locking port 9000	Terminal was forcefully killed with kill -9 without cleanup.	Run the cleanup helper: ./scripts/stop-hadoop-clean.sh

8. Quick Command Cheat Sheet

Task / Action	Command
Install & Configure Hadoop (16-thread)	./install.sh (or double-click setup-wsl.bat)
Start Hadoop & Monitor in Terminal	ran-hadoop
Stop Hadoop Gracefully	**Ctrl + C** in ran-hadoop terminal, or close window
Run WordCount MapReduce Demo	./run-wordcount.sh [filename]
Cluster Health Diagnostics	./scripts/verify-cluster.sh
Emergency Daemon Teardown	./scripts/stop-hadoop-clean.sh
List Files in HDFS	hdfs dfs -ls /
Upload Local File to HDFS	hdfs dfs -put <local-file> <hdfs-path>
Download File from HDFS	hdfs dfs -get <hdfs-path> <local-file>
Read File Content from HDFS	hdfs dfs -cat <hdfs-path>
Check HDFS Cluster Capacity	hdfs dfsadmin -report